Tarea 4: EVIDENCIAS - EJERCICIO FULL-STACK

Integrantes:

David Nickolai Parra Ariza

Jhoan Sebastián Yaya García

Fecha: 04/04/2026

Curso:

2016701-Ingeniería de Software I

Facultad de Ingeniería, Universidad Nacional de Colombia

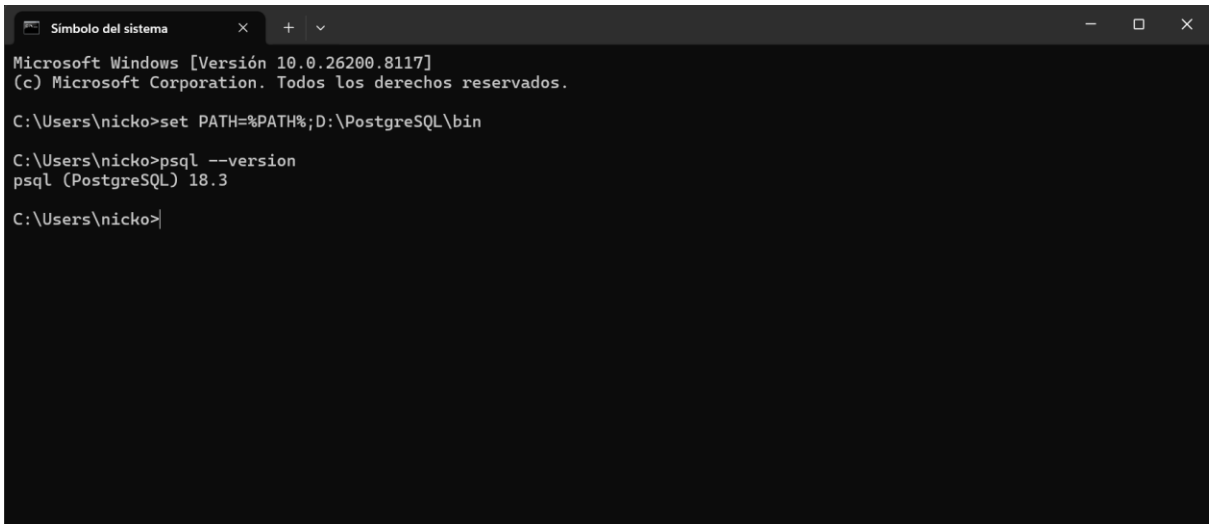
Introducción

El presente documento registra las evidencias del ejercicio de construcción de una aplicación web full-stack, compuesta por un frontend desarrollado en Vue.js, un backend en Django y una base de datos PostgreSQL. El sistema implementa un flujo básico de autenticación (login).

1. Base de datos

Para el almacenamiento de datos se utilizó PostgreSQL 18. A continuación se presenta la verificación de su instalación en el sistema.

Screenshot 1: PostgreSQL instalado



Una vez verificada la instalación, se procedió a crear la base de datos ingesoft1_db y el usuario ingesoft1_user con los permisos necesarios. La siguiente captura confirma la conexión exitosa a la base de datos.

Screenshot 2: Base de datos creada

```
Símbolo del sistema - psql -L

C:\Users\nicko>psql -U ingesoft1_user -d ingesoft1_db -h localhost
Password for user ingesoft1_user:

psql (18.3)
WARNING: Console code page (850) differs from Windows code page (1252)
         8-bit characters might not work correctly. See psql reference
         page "Notes for Windows users" for details.
Type "help" for help.

ingesoft1_db=> \conninfo
        Connection Information
      Parameter       |     Value
----------------------+----------------
 Database             | ingesoft1_db
 Client User          | ingesoft1_user
 Host                 | localhost
 Host Address         | ::1
 Server Port          | 5432
 Options              |
 Protocol Version     | 3.0
 Password Used        | true
 GSSAPI Authenticated | false
 Backend PID          | 16728
 SSL Connection       | false
 Superuser            | off
 Hot Standby          | off
(13 rows)


ingesoft1_db=>
```

Django generó automáticamente la tabla ingesoft1_users en la base de datos mediante el sistema de migraciones. La siguiente captura muestra la estructura de la tabla con sus columnas y tipos de datos.

Screenshot 3: Tabla creada

```
ingesoft1_db=> \d ingesoft1_users
                                  Table "public.ingesoft1_users"
    Column     |          Type          | Collation | Nullable |             Default
---------------+------------------------+-----------+----------+-----------------------------------
 id            | bigint                 |           | not null | generated by default as identity
 correo        | character varying(254) |           | not null |
 password_hash | character varying(128) |           | not null |
 secret_phrase | character varying(255) |           | not null |
Indexes:
    "ingesoft1_users_pkey" PRIMARY KEY, btree (id)
    "ingesoft1_users_correo_6794ae2b_like" btree (correo varchar_pattern_ops)
    "ingesoft1_users_correo_key" UNIQUE CONSTRAINT, btree (correo)


ingesoft1_db=>
```

Se insertó un usuario de prueba en la base de datos con su contraseña almacenada como hash SHA512 y su frase secreta. La siguiente captura confirma que el registro existe correctamente en la tabla.

Screenshot 4: Usuario de prueba

```
ingesoft1_db=> SELECT correo, secret_phrase FROM ingesoft1_users;
            correo             |                              secret_phrase
-------------------------------+-----------------------------------------------------------------------------
 estudiante@universidad.edu.co | ¡Felicitaciones! Has completado el ejercicio de integraci%n Full-Stack.
(1 row)


ingesoft1_db=>
```

2. Backend (Django)

El backend fue desarrollado con Django siguiendo una arquitectura por capas: controllers, services, repositories y domain. La siguiente captura muestra la organización de carpetas del backend siguiendo la arquitectura definida para el ejercicio.

Screenshot 5: Estructura del backend

Nombre	Fecha de modificación	Tipo	Tamaño
__pycache__	4/4/2026 1:48 PM	Carpeta de archivos	
controllers	4/4/2026 1:02 AM	Carpeta de archivos	
domain	4/4/2026 3:53 PM	Carpeta de archivos	
migrations	4/4/2026 1:08 AM	Carpeta de archivos	
repositories	4/4/2026 1:07 AM	Carpeta de archivos	
services	4/4/2026 1:07 AM	Carpeta de archivos	
__init__.py	3/23/2026 10:43 PM	Archivo de origen Py...	0 KB
admin.py	3/23/2026 10:43 PM	Archivo de origen Py...	1 KB
apps.py	3/23/2026 10:43 PM	Archivo de origen Py...	1 KB
tests.py	3/23/2026 10:43 PM	Archivo de origen Py...	1 KB
urls.py	4/4/2026 12:44 AM	Archivo de origen Py...	1 KB
views.py	3/23/2026 10:43 PM	Archivo de origen Py...	1 KB

Con el backend configurado correctamente y la base de datos activa, se procedió a iniciar el servidor de desarrollo de Django. La siguiente captura muestra el servidor corriendo exitosamente en el puerto 8000.

Screenshot 6: Django corriendo

```
(venv) C:\Users\nicko\Desktop\ingesoft1-login\backend>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
April 04, 2026 - 14:56:08
Django version 6.0.3, using settings 'config.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Se verificó el funcionamiento del endpoint POST /api/login mediante una prueba con las credenciales correctas. La siguiente captura muestra la respuesta del servidor ante un login exitoso.

Screenshot 7: API Login exitoso

```
Símbolo del sistema
Microsoft Windows [Versión 10.0.26200.8117]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\nicko>set PATH=%PATH%;D:\PostgreSQL\bin

C:\Users\nicko>curl -X POST http://localhost:8000/api/login -H "Content-Type: application/json" -d "{\"correo\": \"estud
iante@universidad.edu.co\", \"password\": \"MiPassword123\"}"
{"success":true,"secret_phrase":"¡Felicitaciones! Has completado el ejercicio de integración Full-Stack."}
C:\Users\nicko>
```

Se verificó también el comportamiento del endpoint ante credenciales incorrectas, comprobando que el sistema responde adecuadamente con un mensaje de error.

Screenshot 8: API Login fallido

```
C:\Users\nicko>curl -X POST http://localhost:8000/api/login -H "Content-Type: application/json" -d "{\"correo\": \"estud
iante@universidad.edu.co\", \"password\": \"passwordincorrecto\"}"
{"success":false,"message":"Contraseña incorrecta"}
C:\Users\nicko>
```

3. Frontend (Vue.js)

El frontend fue desarrollado con Vue.js utilizando Vite como empaquetador. Se implementó una arquitectura por capas con views, components, router y services. La siguiente captura muestra la organización de carpetas del frontend dentro del directorio src/.

Screenshot 9: Estructura del frontend

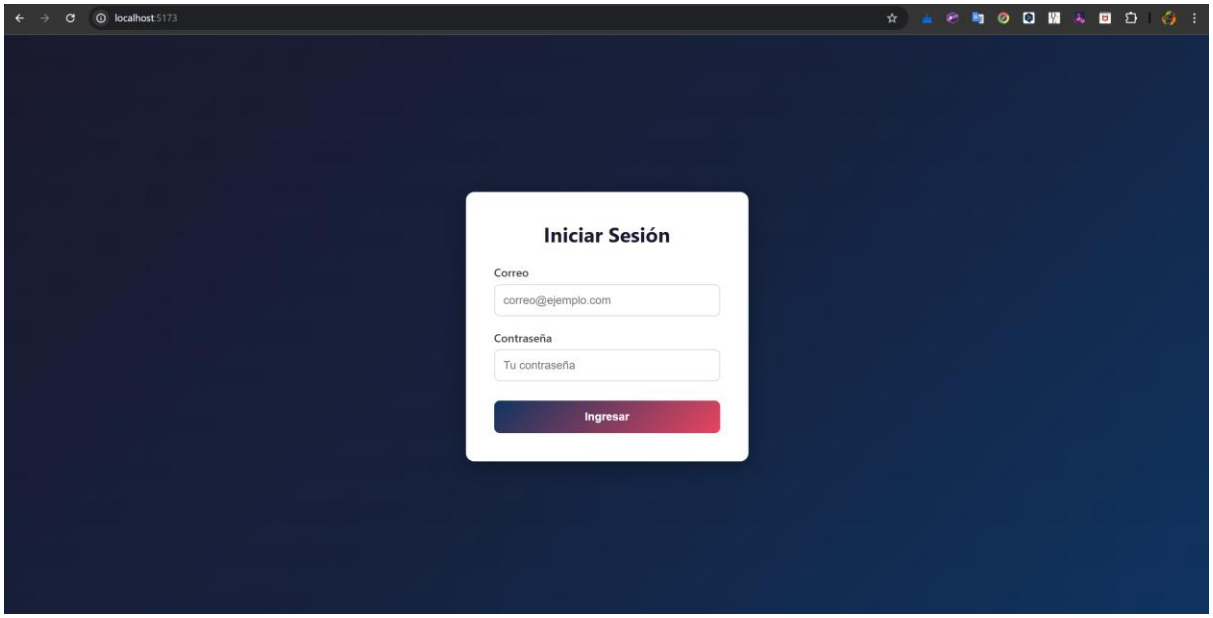
Nombre	Fecha de modificación	Tipo	Tamaño
components	4/2/2026 10:13 PM	Carpeta de archivos	
router	4/2/2026 10:34 PM	Carpeta de archivos	
services	4/2/2026 9:54 PM	Carpeta de archivos	
views	4/2/2026 10:30 PM	Carpeta de archivos	
App.vue	4/2/2026 10:37 PM	Archivo de origen VUE	1 KB
main.js	4/2/2026 10:39 PM	Archivo de origen Jav...	1 KB

Con el frontend configurado correctamente, se inició el servidor de desarrollo de Vite. La siguiente captura muestra el servidor corriendo exitosamente en el puerto 5173.

Screenshot 10: Vue corriendo

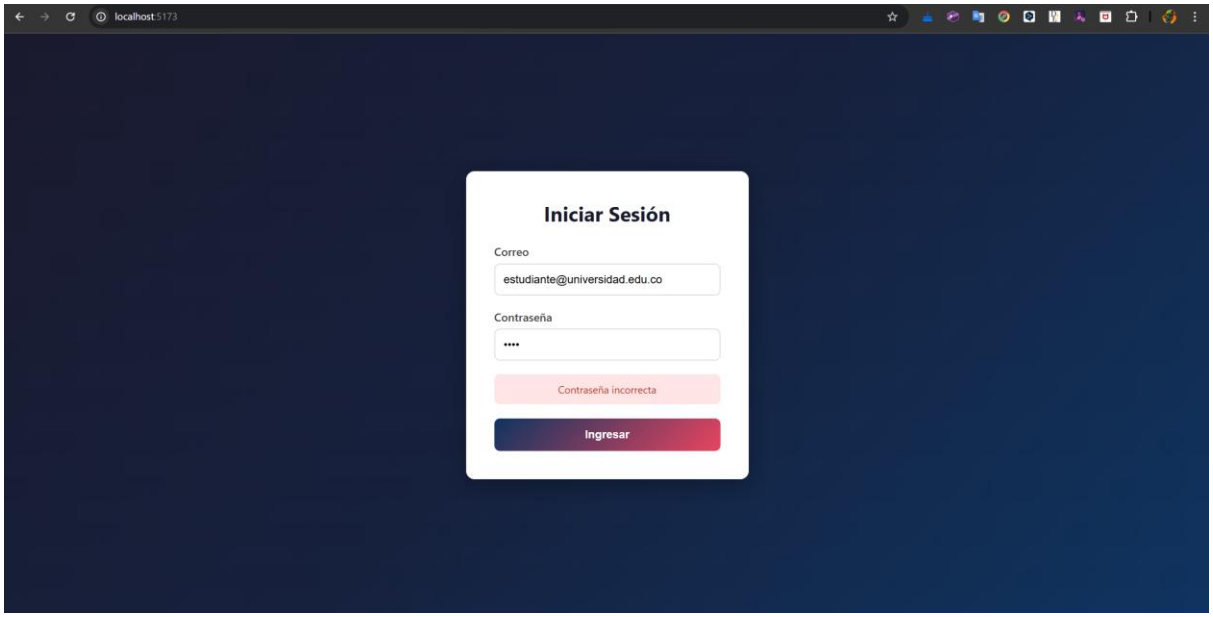
A continuación, se presentan las evidencias del funcionamiento de la interfaz de usuario. La siguiente captura muestra el formulario de login con su diseño final, listo para recibir las credenciales del usuario.

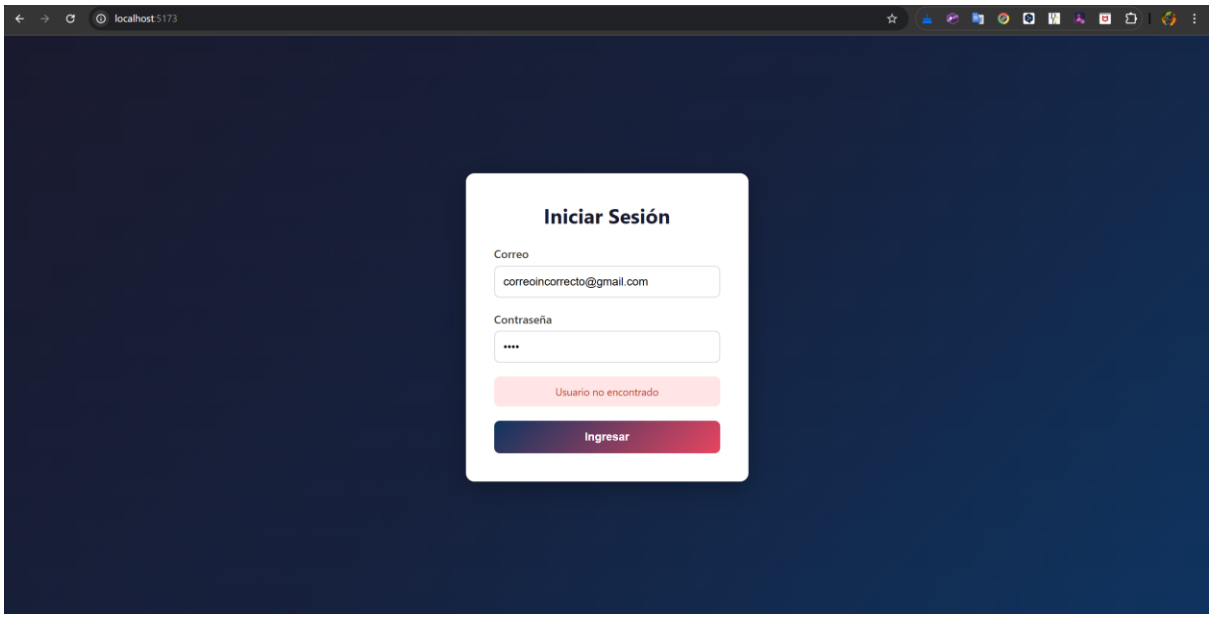
Screenshot 11: Pantalla de login



La siguiente captura muestra el comportamiento del sistema ante el ingreso de credenciales incorrectas, verificando que el mensaje de error se muestra correctamente al usuario.

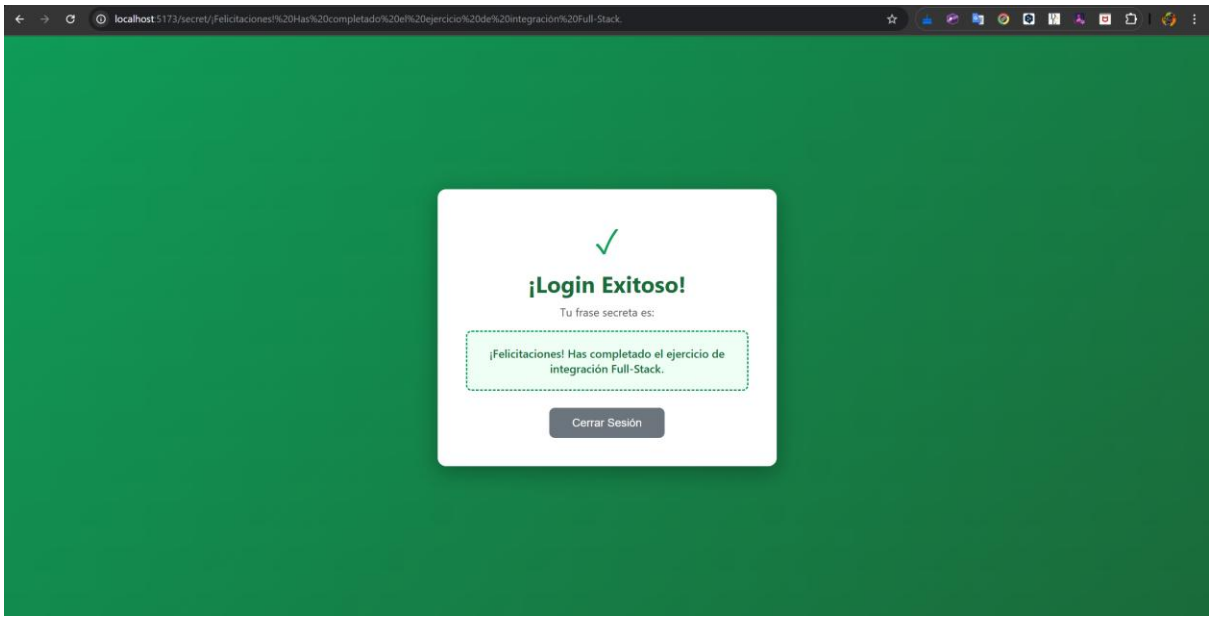
Screenshot 12: Error de login





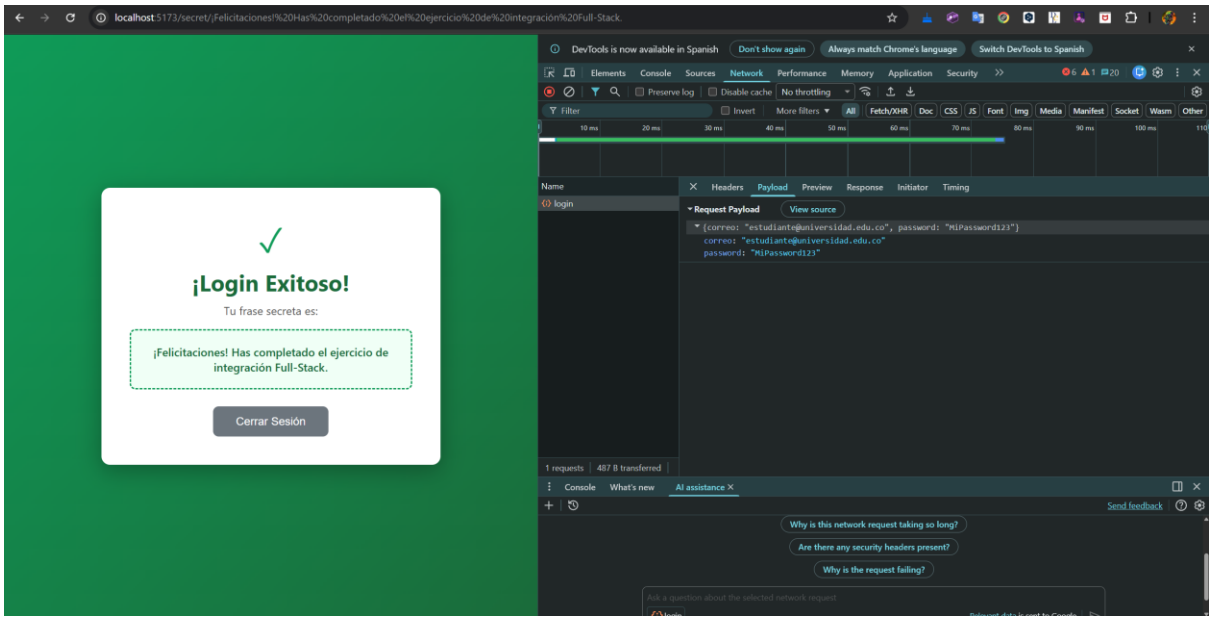
La siguiente captura muestra la pantalla de éxito que se despliega tras ingresar las credenciales correctas, confirmando la integración completa entre frontend, backend y base de datos.

Screenshot 13: Login exitoso



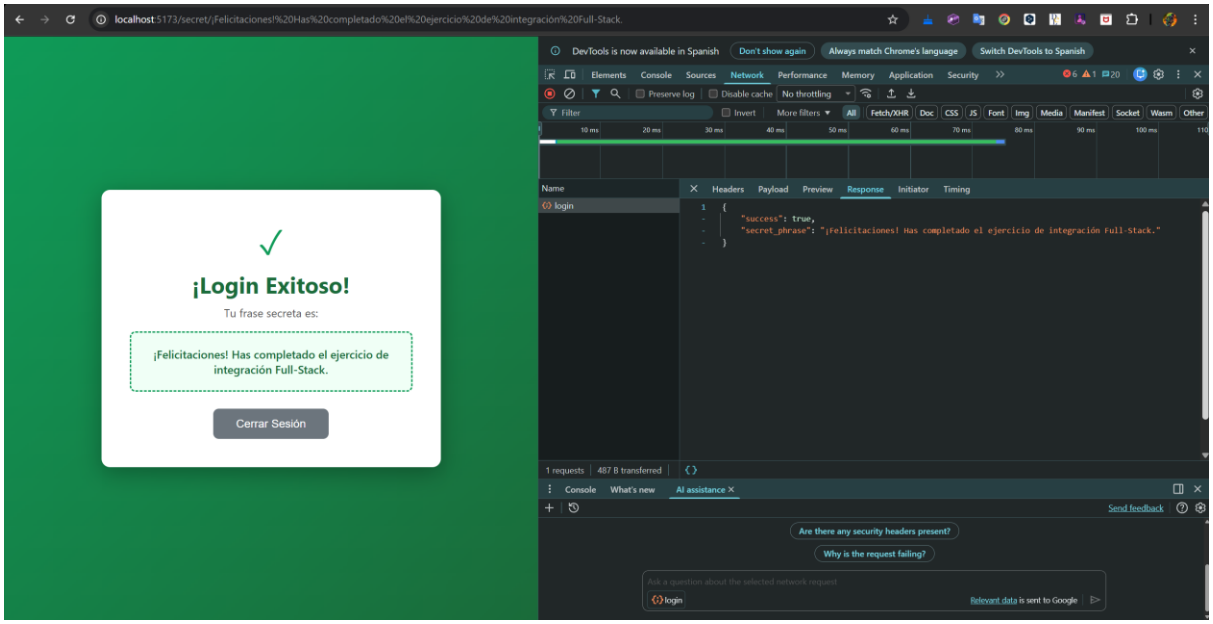
Con los tres componentes del sistema corriendo simultáneamente, se verificó la integración completa mediante las herramientas de desarrollo del navegador. La siguiente captura muestra la petición POST enviada por el frontend al backend durante el proceso de login, verificando que los datos viajan correctamente en formato JSON.

Screenshot 14: DevTools Request



La siguiente captura muestra la respuesta JSON enviada por el backend al frontend tras un login exitoso, confirmando que la comunicación entre las capas del sistema funciona correctamente de extremo a extremo.

Screenshot 15: DevTools Response



TIEMPO TOTAL INVERTIDO:

Aproximadamente 8 horas.